

Asynchronous Secure Document Intelligence Pipeline

A full-stack event-driven medical-document intake and analysis system for VA claims. The browser uploads a medical document, the platform queues work asynchronously, and a Spring Boot worker coordinates validation, document retrieval, AI extraction, and PostgreSQL persistence.

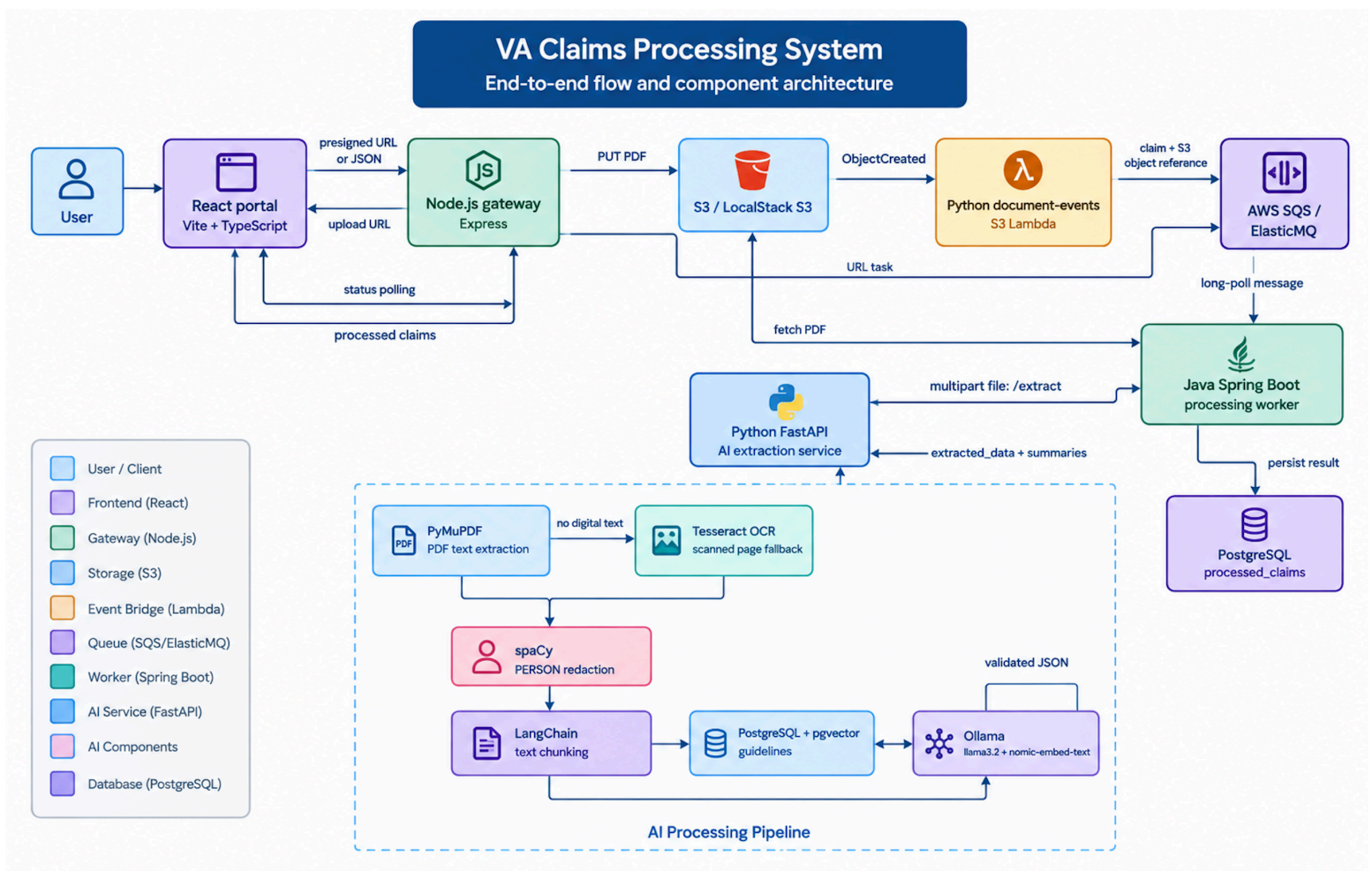
The AI path is isolated in a Python service. It extracts digital PDF text, falls back to OCR for scanned pages, redacts detected person names, chunks the redacted content, retrieves relevant guidelines, and asks a local open-weight model for a strict JSON summary. When AI processing is disabled or unavailable, the worker still persists the claim with a warning status.

Architecture

The system has two supported local upload profiles:

- **LocalStack profile:** the frontend uploads a PDF to LocalStack S3. An S3 ObjectCreated event invokes the Python document-events Lambda, which publishes the S3 reference to LocalStack SQS.
- **URL profile:** the frontend submits document URLs to the Node.js gateway, which publishes a claim task directly to ElasticMQ. This profile is useful for testing the legacy URL contract.

Both profiles converge on the same Spring Boot worker and Python AI extraction service. In AWS, LocalStack and ElasticMQ are replaced by S3, Lambda, and SQS; Terraform provisions the cloud equivalents.



Architectural responsibilities

Component	Responsibility	Local endpoint or port
React portal	Claim identifiers, PDF picker, upload, status refresh, AI summary rendering	`http://localhost:5174` LocalStack profile
Node.js gateway	Request validation, presigned S3 URLs, SQS publishing, status aggregation	`http://localhost:3002` LocalStack profile
Document-events Lambda	Reads S3 metadata and publishes one SQS task per object event	LocalStack internal
Spring Boot worker	SQS consumption, compliance checks, S3 retrieval, AI orchestration, persistence	Internal port `8080`
Python AI service	PDF extraction, OCR fallback, redaction, chunking, retrieval, structured LLM response	`http://localhost:8000`
Ollama	Local open-weight chat and embedding model runtime	`http://localhost:11434`
PostgreSQL	Claim records, extracted fields, summaries, frontend view, guideline embeddings	Internal port `5432`
S3 / LocalStack S3	Original document storage	LocalStack `http://localhost:4566`
SQS / ElasticMQ	Durable asynchronous work buffer and retry boundary	ElasticMQ `9324`

Production-oriented features:

- Asynchronous processing: SQS decouples uploads from heavy document analysis.
- Scalability: workers can be replicated independently from the frontend and gateway.
- Failure handling: SQS retries, dead-letter queues, visibility timeouts, and Resilience4j circuit breaking.
- Graceful degradation: AI failures preserve the claim with COMPLETED_WITH_ANALYSIS_WARNING.
- Input validation: UUID, veteran ID, document URL, filename, content type, and duplicate checks.
- Secure document handling: presigned S3 uploads, metadata-based routing, restricted object paths, and no document contents in logs.
- Privacy controls: spaCy PERSON redaction before LLM processing.
- OCR support: Tesseract fallback for scanned PDFs.
- Structured AI output: Pydantic validation prevents arbitrary LLM responses from entering persistence.
- Database reliability: PostgreSQL, Flyway migrations, JPA transactions, and pgvector retrieval.
- Operational observability: service health endpoints, structured processing logs, queue metrics, status APIs, and error response bodies.
- Deployment repeatability: Docker Compose for local environments and Terraform for AWS infrastructure.

- Configuration controls: environment-driven timeouts, model settings, prompt limits, and an AI enable/disable flag.
- Security middleware: Helmet, CORS, rate limiting, IAM-oriented cloud deployment, and API Gateway JWT support.

That said, it is best described as production-oriented or production-ready in architecture, rather than fully production-ready without further hardening. A real deployment should still add comprehensive automated tests, centralized log aggregation and alerts, secret management validation, backup/restore testing, load testing, stronger authentication coverage, model monitoring, and explicit data-retention policies.

Technology stack and versions

Frontend

- React 18.3.1
- React DOM 18.3.1
- TypeScript 5.3.3
- Vite 7.3.6
- Zod 3.23.8

Gateway

- Node.js runtime: Dockerfile-based Node image, see `apps/3-orchestration-gateway/Dockerfile`
- Express 4.18.2
- AWS SDK S3 3.1126.0
- AWS SDK SQS 3.500.0
- AWS S3 request presigner 3.1126.0
- Helmet 7.1.0
- CORS 2.8.5
- Express rate limit 7.4.0
- TypeScript 5.3.3

Processing worker

- Java 17
- Spring Boot 3.5.6
- Spring Cloud AWS SQS 3.1.0
- AWS SDK for Java S3 2.21.12
- Spring Data JPA
- Flyway database migrations
- Resilience4j Spring Boot 3.2.2.0
- PostgreSQL JDBC driver managed by Spring dependency management

Python AI extraction service

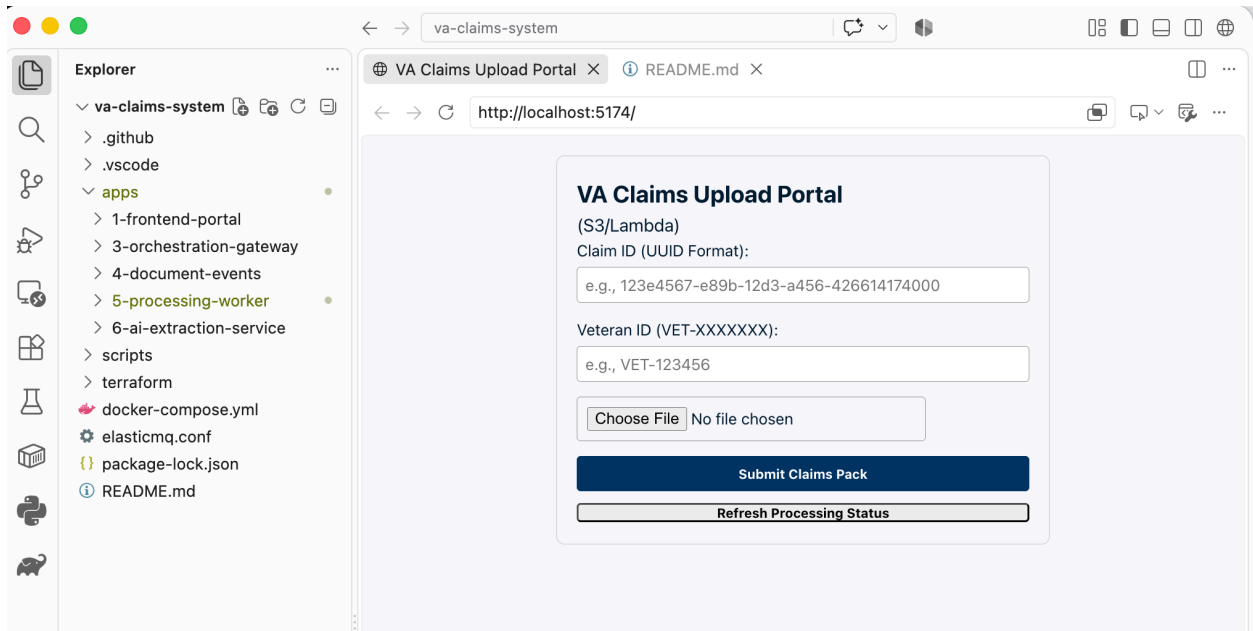
- Python 3.11
- FastAPI 0.115.0
- Uvicorn 0.30.6
- python-multipart 0.0.9
- PyMuPDF 1.24.10
- pytesseract 0.3.13
- Pillow 10.4.0
- spaCy 3.7.5 with en_core_web_sm
- LangChain text splitters 0.2.4
- Pydantic 2.9.2
- HTTPX 0.27.2
- psycpg2-binary 2.9.9
- pgvector Python client 0.3.4
- NumPy 1.26.4

Data, AI, and infrastructure

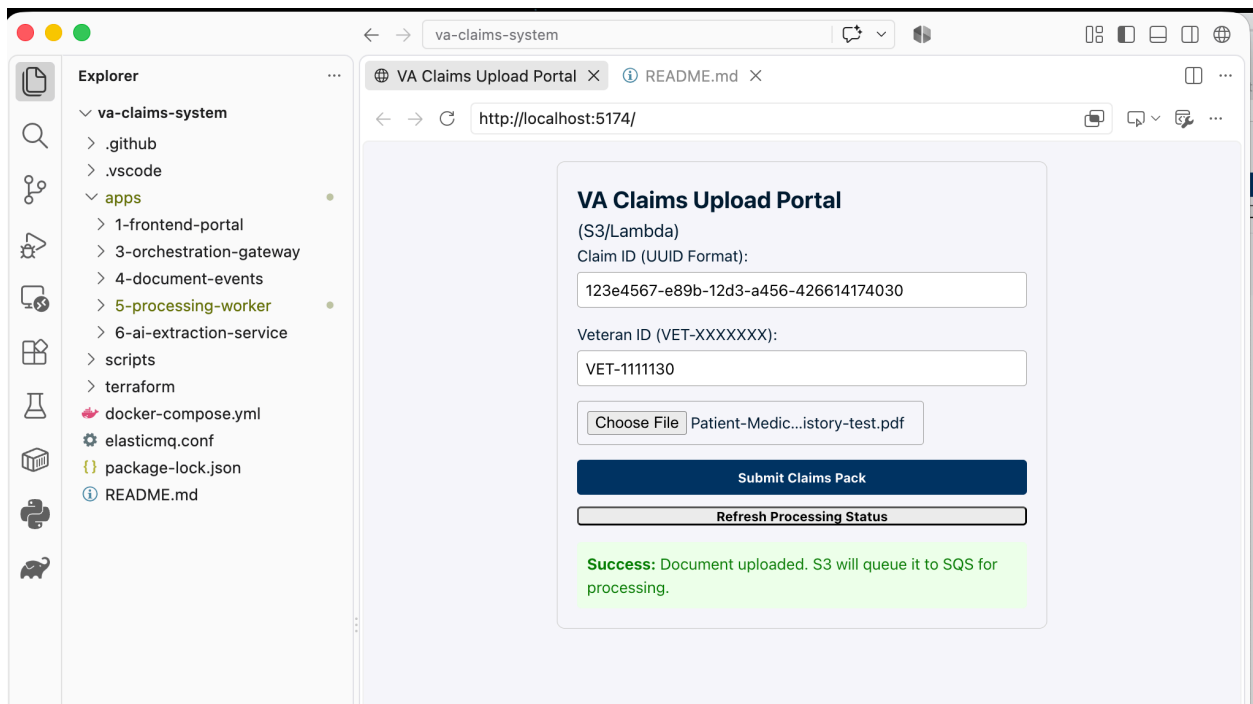
- PostgreSQL 15 with the pgvector extension
- Ollama latest container image
- llama3.2 chat model
- nomic-embed-text embedding model
- Docker Compose
- ElasticMQ 1.6.0 for URL-profile SQS emulation
- LocalStack 3.4 for S3, SQS, and Lambda emulation
- Terraform for AWS infrastructure
- AWS production equivalents: API Gateway, ECS/Fargate, S3, SQS/DLQ, Aurora PostgreSQL, IAM, Secrets Manager, VPC, and CloudWatch

DEMO

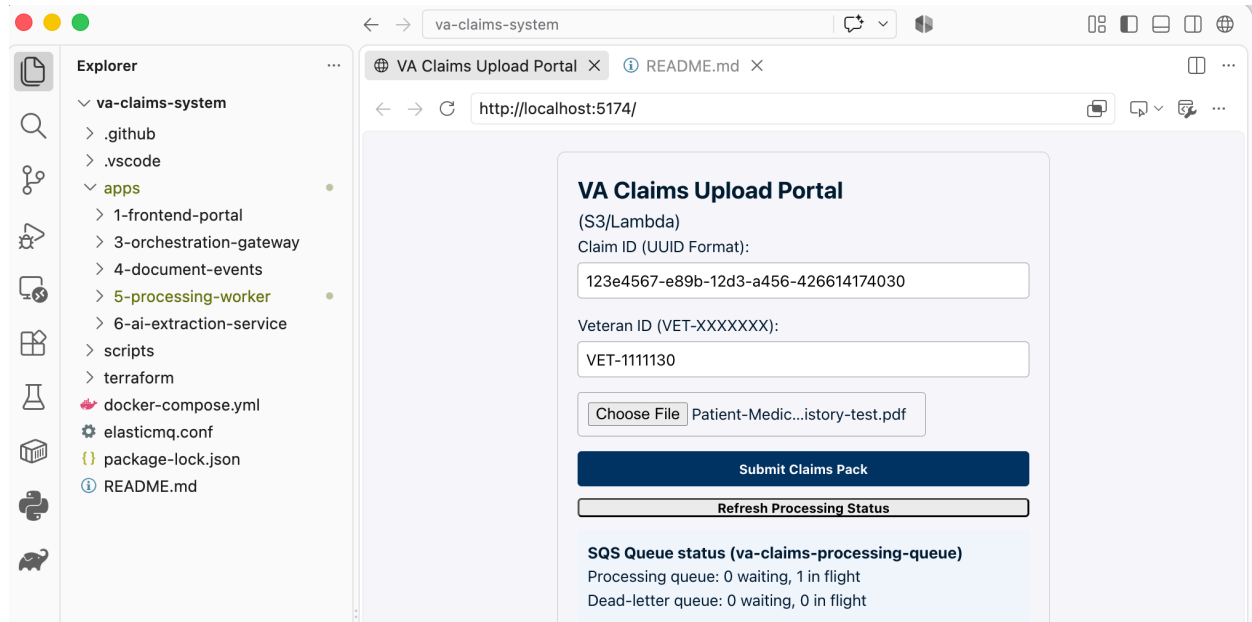
Initial State



Click Submit Claims Pack



Click **Refresh Processing Status** (to see in flight processing)



Click Refresh Processing Status (to see retrieved document summary)

va-claims-system

.github

.vscode

apps

1-frontend-portal

3-orchestration-gateway

4-document-events

5-processing-worker

6-ai-extraction-service

scripts

terraform

docker-compose.yml

elasticsearchmq.conf

package-lock.json

README.md

VA Claims Upload Portal

llm_client.py

http://localhost:5174/

VA Claims Upload Portal

(S3/Lambda)

Claim ID (UUID Format):

123e4567-e89b-12d3-a456-426614174030

Veteran ID (VET-XXXXXX):

VET-11111130

Choose File

Patient-Medic...istory-test.pdf

Submit Claims Pack

Refresh Processing Status

SQS Queue status (va-claims-processing-queue)

Processing queue: 0 waiting, 0 in flight

Dead-letter queue: 0 waiting, 0 in flight

Processed claims

123e4567-e89b-12d3-a456-426614174030 | VET-11111130 | COMPLETED_SUCCESSFULLY

s3://va-claims-local-documents/claims/123e4567-e89b-12d3-a456-426614174030/a634b0d7-83e2-49f0-acf5-b17fd839dbca-Patient-Medical-History-test.pdf

Patient Presentation

New Patient

Acute onset chest pain

Shortness of breath

History of hypertension and peptic ulcer disease

Patient ID: [REDACTED_NAME_1] Chief Complaint: Chest pain

```
processing-worker-localstack-1 | 2026-09-12T03:54:43.902Z INFO 1 ---- [va-processing-worker] [ntContainer#0-1] c.va.worker.consumer.SqsMessageConsumer : [SQS Consumer] Message intercepted. Detected t
type: MedicalDocumentBatch
processing-worker-localstack-1 | 2026-09-12T03:54:43.907Z INFO 1 ---- [va-processing-worker] [ntContainer#0-1] c.va.worker.consumer.SqsMessageConsumer : [SQS Consumer] Processing initialized for Clai
m: 123e4567-e89b-12d3-a456-426614174030 | Veteran: VET-11111130
processing-worker-localstack-1 | 2026-09-12T03:54:43.972Z INFO 1 ---- [va-processing-worker] [ntContainer#0-1] c.v.w.service.DocumentProcessingService : [Heavy Service] Running intensive checks on cl
aim ID: 123e4567-e89b-12d3-a456-426614174030
processing-worker-localstack-1 | 2026-09-12T03:54:44.050Z INFO 1 ---- [va-processing-worker] [ntContainer#0-1] c.va.worker.service.ComplianceService : [Compliance Audit] Claim 123e4567-e89b-12d3-a4
56-426614174030 passed validation for veteran VET-11111130 with 1 documents at 2026-09-12T03:54:44.050020635Z
processing-worker-localstack-1 | 2026-09-12T03:54:44.066Z INFO 1 ---- [va-processing-worker] [ntContainer#0-1] c.va.worker.service.AiExtractionClient : [AI Extraction] Sending document to http://ai-
extraction-service:8000/extract with contentType=multipart/form-data; boundary=-----VaClaimsBoundary138a96fc-39e8-4c3a-a083-a69ff84ef077, boundary=-----VaClaimsBoundary138a96fc-39e8-4c3a-a083-a69ff84ef07
7, documentBytes=90877
ai-extraction-service-1 | INFO:ai_extraction_service:Extracted 15084 chars across 17 chunks (15 names redacted)
ollama-1 | slot get_availabl: id 0 | task -1 | - checking sim = 1.000 (379/379) > 0.100
ollama-1 | slot get_availabl: id 0 | task -1 | selected slot by LCP similarity, f_sim_best = 1.000 (> 0.100 thold), f_keep = 1.000
ollama-1 | slot launch_slot: id 0 | task 2 | processing task, is_child = 0
ollama-1 | slot operator(): id 0 | task 2 | new prompt, n_ctx_slot = 2048, n_keep = 0, task.n_tokens = 379
ollama-1 | slot operator(): id 0 | task 2 | cached n_tokens = 0, memory_seq_rm [0, end)
ollama-1 | slot release: id 0 | task 2 | stop processing: n_tokens = 379, truncated = 0
ollama-1 | srv update_slots: all slots are idle
ollama-1 | INFO:http:HTTP Request: POST http://ollama:11434/api/embeddings "HTTP/1.1 200 OK"
ollama-1 | [GIN] 2026/09/12 - 03:54:45 | 200 | 518.965084ms | 172.21.0.3 | POST "/api/embeddings"
ollama-1 | srv server_stream: conv_id= (empty=1)
ollama-1 | slot get_availabl: id 0 | task -1 | - checking sim = 0.089 (223/2500) > 0.100
ollama-1 | slot get_availabl: id 0 | task -1 | selected slot by LRU, t_last = 15284080024
ollama-1 | srv get_availabl: updating prompt cache
ollama-1 | srv prompt_save: - saving prompt with length 2719, total state size = 297.422 MiB (draft: 0.000 MiB)
ollama-1 | srv load: - looking for better prompt, base f_keep = 0.082, f_sim = 0.089
ollama-1 | srv load: - prompt with length 2719, lcp = 223, f_keep = 0.082, f_sim = 0.089
ollama-1 | srv update: - cache state: 1 prompts, 297.422 MiB (limits: 8192.000 MiB, 4096 tokens, 74890 est)
ollama-1 | srv update: - prompt 0x699d950: 2719 tokens, checkpoints: 0, 297.422 MiB
ollama-1 | srv get_availabl: prompt cache update took 214.61 ms
ollama-1 | slot launch_slot: id 0 | task -1 | sampler chain: logits -> ?penalties -> ?dry -> ?top-n-sigma -> top-k -> ?typical -> ?top-p -> ?min-p -> ?xtc -> temp-ext -> dist
ollama-1 | slot launch_slot: id 0 | task -1 | sampler params:
ollama-1 | load: repeat_last_n = 64, repeat_penalty = 1.000, frequency_penalty = 0.000, presence_penalty = 0.000
ollama-1 | dry_multiplier = 0.000, dry_base = 1.750, dry_allowed_length = 2, dry_penalty_last_n = 64
ollama-1 | top_k = 40, top_p = 1.000, min_p = 0.000, xtc_probability = 0.000, xtc_threshold = 0.100, typical_p = 1.000, top_n_sigma = -1.000, temp = 0.100
ollama-1 | mirostat = 0, mirostat_lr = 0.100, mirostat_ent = 5.000, adaptive_target = -1.000, adaptive_decay = 0.900
ollama-1 | slot launch_slot: id 0 | task 228 | processing task, is_child = 0
ollama-1 | slot operator(): id 0 | task 228 | new prompt, n_ctx_slot = 4096, n_keep = 4, task.n_tokens = 2500
ollama-1 | slot operator(): id 0 | task 228 | cached n_tokens = 223, memory_seq_rm [223, end)
ollama-1 | slot print_timing: id 0 | task 228 | prompt processing, n_tokens = 512, progress = 0.29, t = 3.33 s / 153.83 tokens per second
ollama-1 | slot operator(): id 0 | task 228 | cached n_tokens = 735, memory_seq_rm [735, end)
ollama-1 | slot print_timing: id 0 | task 228 | prompt processing, n_tokens = 1024, progress = 0.50, t = 7.34 s / 139.57 tokens per second
ollama-1 | slot operator(): id 0 | task 228 | cached n_tokens = 1247, memory_seq_rm [1247, end)
ollama-1 | slot print_timing: id 0 | task 228 | prompt processing, n_tokens = 1536, progress = 0.70, t = 11.71 s / 131.18 tokens per second
ollama-1 | slot operator(): id 0 | task 228 | cached n_tokens = 1759, memory_seq_rm [1759, end)
ollama-1 | slot print_timing: id 0 | task 228 | prompt processing, n_tokens = 2048, progress = 0.91, t = 16.61 s / 123.33 tokens per second
ollama-1 | slot operator(): id 0 | task 228 | cached n_tokens = 2271, memory_seq_rm [2271, end)
ollama-1 | slot init_sampler: id 0 | task 228 | init sampler, took 0.71 ms, tokens: text = 2500, total = 2500
```

OCR Fallback Test (in flight)

VA Claims Upload Portal X

← → ↻ http://localhost:5174/

VA Claims Upload Portal

(S3/Lambda)

Claim ID (UUID Format):

123e4567-e89b-12d3-a456-426614174042

Veteran ID (VET-XXXXXX):

VET-11111142

Choose File Medical-Record-Colon-OCR

Submit Claims Pack

Refresh Processing Status

Success: Document uploaded. S3 will queue it to SQS for processing.

Problems 1 Output Debug Console Ports Terminal

processing-worker-localstack-1 | 2026-09-12T06:12:31.404Z INFO 1 --- [va-processing-worker] [ntContainer#0-1] c.va.worker.service.ComplianceService : [Compliance Audit] Claim 123e4567-e89b-12d3-a456-426614174042 passed validation for veteran VET-11111142 with 1 documents at 2026-09-12T06:12:31.404442170Z

processing-worker-localstack-1 | 2026-09-12T06:12:31.433Z INFO 1 --- [va-processing-worker] [ntContainer#0-1] c.va.worker.service.AiExtractionClient : [AI Extraction] Sending document to http://ai-extraction-service:8000/extract with contentType=multipart/form-data; boundary=----VaClaimsBoundary0c557d14-a6c6-40ef-a677-11d10497884f, boundary=----VaClaimsBoundary0c557d14-a6c6-40ef-a677-11d10497884f, documentBytes=980310

ai-extraction-service-1 | INFO:app.extraction:Page 0 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 1 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 2 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 3 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 4 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 5 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 6 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 7 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 8 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 9 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 10 has little/no digital text, falling back to OCR

ai-extraction-service-1 | INFO:app.extraction:Page 11 has little/no digital text, falling back to OCR

1

OCR Fallback Test (complete)

VA Claims Upload Portal

http://localhost:5174/

Refresh Processing Status

SQS Queue status (va-claims-processing-queue)
Processing queue: 0 waiting, 0 in flight
Dead-letter queue: 0 waiting, 0 in flight

Processed claims
123e4567-e89b-12d3-a456-426614174042 | VET-11111142 | COMPLETED_SUCCESSFULLY
s3://va-claims-local-documents/claims/123e4567-e89b-12d3-a456-426614174042/a4965b62-1683-46eb-ba29-b47636f56826-Medical-Record-Colon-OCR.pdf

Colon Cancer Diagnosis

- Patient's history of diabetes mellitus, hypertension, and hypercholesterolemia
- Patient's newly diagnosed colon cancer status post right-sided colectomy
- Patient's T3-NO-MX adenocarcinoma of the colon status post right-sided colectomy

123e4567-e89b-12d3-a456-426614174041 | VET-11111141 | COMPLETED_WITH_ANALYSIS_WARNING
s3://va-claims-local-documents/claims/123e4567-e89b-12d3-a456-426614174041/d935efda-ecc5-48f7-93dd-b884ce12cf51-Medical-Record-Colon-OCR.pdf

Problems 1

Output

Debug Console

Ports

Terminal

```
ai-extraction-service-1 | INFO:app.extraction:Page 7 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 8 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 9 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 10 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 11 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 12 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 13 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:app.extraction:Page 14 has little/no digital text, falling back to OCR
ai-extraction-service-1 | INFO:ai_extraction_service:Extracted 19086 chars across 21 chunks (31 names redacted)
ollama-1 | time=2026-09-12T06:12:46.429Z level=INFO source=sched.go:1147 msg="disabling mmap for llama-server load by default" model=/root/.ollama/models/blobs/sha256-970aa74c0a90ef7482477cf803618e776e173c007bf957f635f1015bfcfef0e6 reason=cpu
ollama-1 | time=2026-09-12T06:12:46.429Z level=INFO source=server.go:109 msg="using llama-server for model" model=/root/.ollama/models/blobs/sha256-970aa74c0a90ef7482477cf803618e776e173c007bf957f635f1015bfcfef0e6
ollama-1 | time=2026-09-12T06:12:46.429Z level=WARN source=server.go:114 msg="requested context size too large for model" num_ctx=8192 n_ctx_train=2048
ollama-1 | time=2026-09-12T06:12:46.430Z level=INFO source=llama_server.go:433 msg="starting llama-server" cmd="/usr/lib/ollama/llama-server --model /root/.ollama/models/blobs/sha256-970aa74c0a90ef7482477cf803618e776e173c007bf957f635f1015bfcfef0e6 --port 34963 --host 127.0.0.1 --no-webui --offline -c 2048 -np 1 --log-verbosity 4 --no-log-prefix --no-log-timestamps --load-mode none --flash-attn auto --embedding -b 2048 -ub 2048 --context-shift --keep 4"
```

1

Gordon

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

MCP Toolkit

Docker Hub

Extensions

PERSONAL

Q Search

36K

C

Containers

Give feedback

Container CPU usage

10.64% / 1000% (10 CPUs available)

Container memory usage

6.4GB / 7.57GB

Show charts

Q Search

Only show running containers

		Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐		enterprise_postgres	0c93690a9133	postgres:1	5433:5432	0%	
☐		enterprise_qdrant	0ab07814093b	qdrant/qdr	6333:6333	0.13%	
☐	>	langfuse	-	-	-	9.46%	
☐	▼	va-claims-system	-	-	-	0.99%	
☐		local-sqs-1	db86de062f5b	softwarem	9324:9324	0.33%	
☐		processing-worker-1	901af51d1cb2	va-claims-s	Show all ports (2)	0.18%	
☐		orchestration-gateway-1	2d288a1f7372	va-claims-s	3001:3000	0%	
☐		frontend-portal-1	d687e7348081	va-claims-s	5173:80	0%	
☐		local-postgres-1	213c534f993f	pgvector/p		0%	
☐		frontend-portal-localstack-1	e7e759f8f6dd	va-claims-s	5174:80	0%	
☐		orchestration-gateway-localstack-1	ebbe1b3665c2	va-claims-s	3002:3000	0%	
☐		localstack-1	942b6501e838	va-claims-s	4566:4566	0.08%	
☐		ollama-1	cc1fd7fe6710	ollama/oll	11434:11434	0.01%	
☐		ai-extraction-service-1	867d40f87d02	va-claims-s	8000:8000	0.21%	

Processing sequence

1. User enters a UUID claim ID and VET-prefixed veteran ID in the React portal.
2. User selects a PDF medical document.
3. The portal requests a presigned upload URL from the Node gateway.
4. The portal uploads the PDF directly to S3 or LocalStack S3.
5. S3 emits an ObjectCreated event.
6. The document-events Lambda reads claim-id, veteran-id, and content metadata.
7. The Lambda publishes a MedicalDocumentBatch message to SQS.
8. The Spring worker long-polls SQS and receives the task.
9. ComplianceService validates the claim and document reference.
10. AiExtractionClient downloads the PDF from S3.
11. AiExtractionClient sends a multipart form field named `file` to POST /extract.
12. FastAPI extracts digital PDF text with PyMuPDF.
13. Pages without usable text are processed with Tesseract OCR.
14. spaCy replaces PERSON entities with redaction placeholders.
15. LangChain splits redacted text into bounded overlapping chunks.
16. The AI service retrieves matching guideline embeddings from PostgreSQL/pgvector.
17. Ollama generates structured JSON using llama3.2.
18. Pydantic validates extracted_data, database_summary, and frontend_view.
19. The AI response returns to the Spring worker.
20. The worker persists the result in processed_claims and deletes the SQS message.
21. The gateway reads processed claims from the worker status endpoint.
22. The portal refreshes and renders the AI summary card.

Processing Flow Details

This section maps each processing-sequence step to the class, method, endpoint, or service that performs it.

1. Capture claim identifiers

The React component `ClaimUploadForm` stores the claim ID, veteran ID, selected file, and optional URL input in component state. The form's `handleSubmit` method builds the upload payload and chooses the S3 upload path when `VITE_S3_UPLOAD_ENABLED=true`; otherwise it chooses the URL submission path.

- **Success log/state:** the form proceeds after valid UUID and veteran ID input.
- **Failure:** the portal displays Zod validation messages; the gateway may return `Claim ID must be a valid UUID.` or `Veteran ID must match VET- followed by 6 to 10 digits.`

2. Select a PDF document

In S3 mode, the file input in `ClaimUploadForm` accepts the document and passes the selected `File` to `useClaimUpload.uploadDocument`. The AI service currently supports PDF documents for extraction, even though the gateway upload contract also permits JSON, JPEG, and PNG objects for broader storage/event testing.

- **Success log/state:** the browser calls `uploadDocument` and requests an upload URL.
- **Failure:** the portal displays `Unable to prepare document upload. or A valid document filename is required.;` the AI service returns `Only PDF documents are supported for a non-PDF extraction request.`

3. Request an upload URL or submit a URL task

`useClaimUpload.uploadDocument` sends `POST /api/v1/claims/upload-url` to the gateway with the claim ID, veteran ID, filename, and content type. For the legacy URL path, `useClaimUpload.submitClaimData` validates the browser payload with `ClaimUploadSchema` and sends `POST /api/v1/claims/upload`.

- **Success log/state:** the gateway returns HTTP 200 for `/upload-url` or HTTP 202 with `PROCESSING_QUEUED` for `/upload`.

- **Failure:** the portal displays Unable to prepare document upload. or Server rejected processing request.; the gateway can return Document storage is not configured. or Failed to safely queue processing task. Please try again later.

4. Validate the gateway request

`UploadController.createUploadUrl` normalizes identifiers and filename data, validates the UUID, veteran ID, filename, and content type, creates an S3 key under `claims/<claim-id>/`, and returns a presigned URL with claim metadata headers.

`UploadController.handleClaimUpload` performs equivalent validation for URL submissions before creating a `HeavyProcessingPayload`.

- **Success log/state:** a presigned URL and object key are returned.
- **Failure:** HTTP 400 identifies an invalid claim ID, veteran ID, filename, content type, or request payload; HTTP 503 indicates document storage is not configured.

5. Upload the document to storage

`useClaimUpload.uploadDocument` performs a browser PUT to the returned presigned URL. The S3 object contains `claim-id` and `veteran-id` metadata. The browser does not send the document through the gateway, keeping the synchronous request small and moving file transfer to object storage.

- **Success log:** LocalStack logs an S3 PutObject request with HTTP 200; the portal displays Document uploaded. S3 will queue it to SQS for processing.
- **Failure:** the portal displays S3 document upload failed.; a missing or invalid object event prevents the next Lambda log.

6. Receive the S3 ObjectCreated event

LocalStack or AWS S3 invokes `lambda_handler` in `apps/4-document-events/src/handler.py`. The handler reads the bucket and URL-decoded object key from the event, then calls `head_object` to retrieve object metadata and content type.

- **Success log:** LocalStack logs the S3 ObjectCreated/Lambda invocation and the handler proceeds to `head_object`.
- **Failure:** the Lambda raises S3 object <key> is missing claim-id or veteran-id metadata when required metadata is absent.

7. Publish the SQS task

`lambda_handler` creates a `MedicalDocumentBatch` message containing `claimId`, `veteranId`, `submittedAt`, `bucket`, `objectKey`, and `contentType`, then calls `sqs.send_message`. It adds the `MessageType` attribute expected by the Java listener. The URL profile takes a parallel route through `SqsService.queueHeavyProcessingTask`, which calls AWS SDK `SendMessageCommand`.

- **Success log:** the gateway logs [SQS Success] Message offloaded safely. MessageId: ...; the Lambda returns `{"queued": 1}`.
- **Failure:** the gateway logs [SQS Failure] Failed to send message to queue;; the Lambda can fail if SQS is unavailable or the queue URL is invalid.

8. Long-poll and deserialize the task

`SqsMessageConsumer.receiveMessage` is registered with `@SqsListener("${aws.sqs.queue-name}")`. Spring Cloud AWS continuously polls SQS. It logs the message type, uses `Jackson ObjectMapper.readValue` to create `ClaimTaskDto`, and delegates the task to `DocumentProcessingService.processMedicalBatch`. If the method returns normally, Spring deletes the message; if an exception escapes, the message remains available for retry after the visibility timeout.

- **Success log:** [SQS Consumer] Message intercepted. Detected type: MedicalDocumentBatch followed by [SQS Consumer] Processing initialized for Claim:

- **Failure:** [SQS Consumer Error] Failed to process payload. Message returned to queue for retry.; malformed JSON or a missing message attribute can cause this path.

9. Run compliance validation

`ComplianceService.validate` checks that the task exists, the claim ID is a UUID, the veteran ID matches `VET-` followed by 6 to 10 digits, and `submittedAt` is a valid non-future ISO-8601 instant. S3 tasks use `verifyS3DocumentIsAccessible` and `S3Client.headObject`; URL tasks use `validateDocuments`, `parseDocumentUri`, and `verifyDocumentIsAccessible` with an HTTP HEAD request. Invalid tasks throw `ComplianceValidationException` before persistence.

- **Success log:** [Compliance Audit] Claim ... passed validation for veteran ... with ... documents at
- **Failure:** `ComplianceValidationException`, such as Claim ID must be a valid UUID, At least one medical document is required, Medical document is not accessible: HTTP ..., or S3 medical document could not be reached:

10. Fetch the original PDF

`DocumentProcessingService.processMedicalBatch` calls `AiExtractionClient.extract`. The client calls `fetchDocumentBytes`, which uses `S3Client.getObject` with the task bucket and object key and reads the object into memory. Empty or missing S3 references produce an unavailable AI result rather than crashing claim persistence.

- **Success log:** the later multipart message includes `documentBytes=...`, proving non-empty bytes were fetched.
- **Failure:** Claim task has no S3-referenced document, Claim task document is empty, or Unable to read document or AI extraction response.

11. Send the multipart `/extract` request

`AiExtractionClient.callAiService` creates a fixed-length multipart request. `buildMultipartBody` emits a file part with a sanitized filename, `Content-Type: application/pdf`, the PDF bytes, and a closing boundary. The client sends the request to `POST <AI_EXTRACTION_SERVICE_URL>/extract` and logs only the endpoint, content type, boundary, and byte count. It does not log document contents.

- **Success log:** [AI Extraction] Sending document to `http://ai-extraction-service:8000/extract` with `contentType=multipart/form-data; boundary=...` `documentBytes=...`
- **Failure:** HTTP 422 with `body.file Field required`, HTTP 400 with `Only PDF documents are supported`, or HTTP 502 with `Document analysis failed: ....`

12. Extract digital PDF text

FastAPI routes `POST /extract` to `app.main.extract`. The endpoint validates the required `UploadFile` field named `file`, checks that the content type is PDF or `octet-stream`, reads the bytes, and calls `extract_text_from_pdf` in `app/extraction.py`. That function opens the document with `fitz`, reads page text, and retains digital text when a page contains at least 20 characters.

- **Success log:** `Extracted ... chars across ... chunks (... names redacted)` after extraction continues.
- **Failure:** malformed PDFs or extraction exceptions are logged as `Document analysis failed` and returned as HTTP 502.

13. Fall back to OCR

For pages with fewer than 20 digital characters, `extract_text_from_pdf` renders the page at `OCR_RENDER_ZOOM=2.0`, converts it to a PIL image, and calls `pytesseract.image_to_string`. Digital and OCR text are joined into one document string for downstream processing.

- **Success log:** Page `<zero-based-page-number>` has little/no digital text, falling back to OCR.
- **Failure:** OCR exceptions are included in `Document analysis failed: ...` and the service traceback.

14. Redact person names

`app/redaction.py` lazily loads spaCy's `en_core_web_sm` pipeline through `_get_pipeline`. `redact_names` scans the extracted text for PERSON entities and replaces each name with sequential placeholders such as `[REDACTED_NAME_0]`. It returns both the redacted text and the number of replacements for service logging.

- **Success log:** Extracted 15084 chars across 17 chunks (15 names redacted) or the equivalent counts. Names and document contents are intentionally never logged.
- **Failure:** a missing spaCy model can produce `[E050] Can't find model 'en_core_web_sm', wrapped as Document analysis failed: ....`

15. Chunk the redacted document

`app/chunking.py` uses a `RecursiveCharacterTextSplitter` configured with 1,200-character chunks and 150-character overlap. `chunk_text` returns bounded chunks using paragraph, line, sentence, word, and character separators. The overlap preserves context across chunk boundaries.

- **Success log:** the aggregate extraction log reports the number of chunks, for example `Extracted ... chars across 17 chunks ....`
- **Failure:** chunking exceptions are logged under `Document analysis failed`; an empty document produces zero chunks and may lead to an unusable model response.

16. Retrieve applicable guidelines

`summarize_and_extract` calls `retrieve_guidelines` with the first 2,000 characters of the redacted document. `guidelines._embed` requests an embedding from Ollama's `/api/embeddings` endpoint using `nomic-embed-text`. `retrieve_guidelines` registers pgvector, orders non-null guideline rows by vector distance using `embedding <=> %s::vector`, and falls back to `DEFAULT_SYSTEM_GUIDELINES` if retrieval is disabled or unavailable. `ensure_guideline_embeddings`, called by the FastAPI startup hook, fills missing guideline embeddings.

- **Success log:** Ollama logs `POST /api/embeddings` with HTTP 200; the AI service uses retrieved guidelines.
- **Failure:** Embedding request failed; guideline retrieval will use defaults, Guideline retrieval query failed; using defaults, or Unable to backfill guideline embeddings. These are fallback warnings, not necessarily fatal errors.

17. Generate structured output with Ollama

`llm_client.summarize_and_extract` builds `SYSTEM_PROMPT_TEMPLATE` with the retrieved guidelines and combines a bounded number of redacted chunks into the user prompt. The configured limits are `LLM_MAX_PROMPT_CHARS=9000` and `LLM_MAX_PROMPT_CHUNKS=12`. It sends an OpenAI-compatible `POST /v1/chat/completions` request to Ollama with model `llama3.2`, a strict `ExtractionResponse.model_json_schema()` response contract, low temperature, and `num_ctx=LLM_CONTEXT_SIZE` (2048 by default). The prompt forbids wrapper objects such as `{ "json": ... }`. `frontend_view.metrics` accepts string, integer, and floating-point values so count-like metrics do not fail validation.

- **Success log:** Ollama logs `POST /v1/chat/completions` with HTTP 200.
- **Failure:** `httpx.ReadTimeout: timed out`, prompt truncation, or model-loading messages; the AI service returns HTTP 502 with `Document analysis failed: timed out`.

18. Validate the JSON response

The LLM response content is parsed with `json.loads` and validated by `ExtractionResponse.model_validate`. The Pydantic schema requires `extracted_data`, `database_summary`, and `frontend_view`; it also validates badge colors and the extracted condition, diagnosis, treatment-date, and provider fields. Invalid JSON or schema output raises an error from the AI endpoint.

- **Success log/state:** FastAPI returns HTTP 200 for `/extract` and the worker continues to persistence.

- **Failure:** `LLM response failed schema validation: ...`; FastAPI returns HTTP `502` with the validation error. Repeated invalid responses count as AI-call failures and can open the Resilience4j `aiExtractionService` circuit breaker, producing `CircuitBreaker 'aiExtractionService' is OPEN and does not permit further calls` until the wait period expires or the worker is restarted.

19. Return the AI result to Java

After FastAPI returns the validated response, `AiExtractionClient.callAiService` parses the JSON with Jackson and serializes the `extracted_data` and `frontend_view` nodes for database storage. HTTP errors retain the status and response body in the worker log. The Resilience4j circuit breaker around `extract` converts connection, timeout, and service failures into an unavailable result.

- **Success log:** [AI Extraction] Claim ... analyzed (successful=true, skipped=false).
- **Failure:** [AI Extraction] Failed to analyze document for claim ...: AI extraction service responded with HTTP ...: ...,Unable to read document or AI extraction response, or a circuit-breaker failure reason.

20. Persist the processed claim

`DocumentProcessingService.processMedicalBatch` creates `ClaimEntity`, sets the identifiers, S3 path or URL path, gateway submission time, completion time, extracted data, database summary, and frontend view, then calls `ClaimRepository.save`. A successful or intentionally skipped AI result receives `COMPLETED_SUCCESSFULLY`; an unavailable AI result receives `COMPLETED_WITH_ANALYSIS_WARNING`. The surrounding `@Transactional` method commits the row before the SQS listener completes.

- **Success log:** [Database Persist] Successfully committed record to Aurora. Internal DB key: ...; status is `COMPLETED_SUCCESSFULLY`.
- **Failure or degraded completion:** AI failure still persists `COMPLETED_WITH_ANALYSIS_WARNING`; database or transaction failures prevent the persist log and cause SQS retry.

21. Read queue and processed-claim status

The frontend's `useClaimUpload.getProcessingStatus` calls `GET /api/v1/claims/status`.

`UploadController.getProcessingStatus` delegates to `SqsService.getQueueStatus`, which reads visible and in-flight counts with `GetQueueAttributesCommand`. When `WORKER_STATUS_URL` is configured, the same method fetches `/internal/claims/processed` from `ClaimStatusController.processedClaims` and includes the latest 20 persisted claims.

- **Success log/state:** `GET /api/v1/claims/status` returns HTTP 200 with queue counts and `processedClaims`.
- **Failure:** [Status Error] Failed to read SQS status: and HTTP 503 with `Processing status is temporarily unavailable`.

22. Render the result in the portal

`ClaimUploadForm.refreshProcessingStatus` stores the status response and renders queue counts, system status, and processed claims. For each claim, `AiSummaryCard` parses the `frontendView` JSON and displays the title, colored badges, bullet summary, and metrics. An empty or invalid `frontendView` is intentionally rendered as no summary card, which is why `COMPLETED_WITH_ANALYSIS_WARNING` generally has no AI summary.

- **Success state:** the portal displays the claim under `Processed claims`, and `AiSummaryCard` renders the title, badges, bullets, and metrics from `frontendView`.
- **Failure or no summary:** `COMPLETED_WITH_ANALYSIS_WARNING`, an empty `{}` frontend view, malformed JSON, or a missing title results in no AI summary card.

AI processing controls

The local defaults are designed for CPU-based demos:

```
AI_EXTRACTION_ENABLED=true
AI_EXTRACTION_TIMEOUT_SECONDS=300
LLM_TIMEOUT_SECONDS=300
LLM_MAX_PROMPT_CHARS=9000
LLM_MAX_PROMPT_CHUNKS=12
LLM_CONTEXT_SIZE=4096
SQS_visibility_timeout=600 seconds
```

To preserve the legacy persistence-only behavior:

```
AI_EXTRACTION_ENABLED=false docker compose --profile localstack up --build
```

Successful AI processing produces `COMPLETED_SUCCESSFULLY`. A persisted claim whose AI call fails produces `COMPLETED_WITH_ANALYSIS_WARNING`; the original claim data remains available, but the AI summary fields are empty.

Local deployment

Prerequisites

- Docker Desktop with the Docker daemon running
- Docker Compose plugin
- A PDF document for the upload demo

Start the full AI demo

From the repository root:

```
docker compose --profile localstack up --build
```

Open the frontend at `<http://localhost:5174>`. Enter a valid UUID, for example `123e4567-e89b-12d3-a456-426614174000`, enter a veteran ID such as `VET-123456`, select a PDF, and click **Submit Claims Pack**. After processing completes, click **Refresh Processing Status**.

The first run downloads the Ollama models. Watch progress with:

```
docker compose logs -f ollama-model-init
```

The active processing logs are:

```
docker compose logs -f processing-worker-localstack ai-extraction-service ollama
```

Successful processing should include:

```
POST /extract HTTP/1.1 200 OK
[AI Extraction] Claim ... analyzed (successful=true, skipped=false)
[Database Persist] Successfully committed record ...
```

Check service status:

```
docker compose ps
```

Stop the stack:

```
docker compose down
```

Stop it and remove local database/model volumes:

```
docker compose down -v
```

Health and status endpoints

```
curl http://localhost:8000/health
curl http://localhost:11434/api/tags
curl http://localhost:3002/health
curl http://localhost:3002/api/v1/claims/status
```

The internal worker status endpoint is:

```
GET /internal/claims/processed
```

It returns recent claim IDs, veteran IDs, document references, system statuses, extracted data, database summaries, and frontend view JSON.

URL-based local profile

The URL profile uses ElasticMQ and the gateway's legacy JSON contract:

```
docker compose --profile local-url up --build
```

Open <http://localhost:5173> and provide a reachable HTTP(S) document URL. The gateway publishes the task directly to ElasticMQ. This path is useful for testing validation and retry behavior, but the Python AI service requires the worker to receive an S3-backed document for /extract.

API contracts

S3 upload URL request

```
{
  "claimId": "123e4567-e89b-12d3-a456-426614174000",
  "veteranId": "VET-123456",
  "fileName": "medical-document.pdf",
  "contentType": "application/pdf"
}
```

S3 upload URL response

```
{
  "uploadUrl": "http://localhost:4566/...",
  "objectKey": "claims/<claim-id>/<document-id>-medical-document.pdf",
  "expiresIn": 300
}
```

Processed claim response shape

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "veteranId": "VET-123456",
  "documentPaths": "s3://va-claims-local-documents/claims/...",
  "systemStatus": "COMPLETED_SUCCESSFULLY",
  "extractedData": "{\"conditions\":[],\"diagnoses\":[]}",
  "databaseSummary": "Concise factual summary",
  "frontendView": "{\"title\":\"Medical Document Summary\", \"badges\":[], \"bullets\":[], \"metrics\":{\"}}"
```

Project structure

```
.
|-- apps/                                Application services
|   |-- 1-frontend-portal/              React/Vite claims portal
|   |   |-- src/App.tsx                 Main portal composition
|   |   |-- src/main.tsx                React application entry point
|   |   |-- src/assets/theme.css        Shared portal styling
|   |   |-- src/features/claims/
|   |   |   |-- components/ClaimUploadForm.tsx Upload form and status display
|   |   |   |-- components/AiSummaryCard.tsx  AI frontend_view renderer
|   |   |   |-- hooks/useClaimUpload.ts       Upload, queue, and status API calls
|   |   |   |-- types/claim.validation.ts     Zod claim input validation
|   |   |   |-- types/frontendView.ts        TypeScript AI summary contract
|   |   |-- Dockerfile                  Production static asset image
|   |   |-- package.json                Frontend dependencies and scripts
|   |   |-- index.html                  Vite HTML entry point
|   |-- 3-orchestration-gateway/         Node.js orchestration gateway
|   |   |-- src/app.ts                  Express routes and middleware
|   |   |-- src/controllers/upload.controller.ts Upload and status endpoints
|   |   |-- src/services/sqs.service.ts    SQS publishing and queue metrics
|   |   |-- Dockerfile                  Gateway runtime image
|   |   |-- package.json                Node dependencies and scripts
|   |   |-- tsconfig.json               TypeScript compiler settings
|   |-- 4-document-events/              S3 event bridge
|   |   |-- src/handler.py              S3 event Lambda handler
|   |   |-- localstack-init.sh          Local S3, SQS, and Lambda setup
|   |   |-- Dockerfile                  LocalStack image and Lambda package
|   |-- 5-processing-worker/            Spring Boot background worker
|   |   |-- src/main/java/com/va/worker/
|   |   |   |-- WorkerApplication.java      Spring Boot application entry point
|   |   |   |-- consumer/SqsMessageConsumer.java SQS listener and task deserializer
|   |   |   |-- controller/ClaimStatusController.java Internal claim status endpoint
|   |   |   |-- model/ClaimEntity.java      Processed claim JPA entity
|   |   |   |-- repository/ClaimRepository.java JPA persistence repository
|   |   |   |-- service/ComplianceService.java Claim and document validation
|   |   |   |-- service/DocumentProcessingService.java Processing orchestrator
|   |   |   |-- service/AiExtractionClient.java S3 fetch and AI service client
|   |   |-- src/main/resources/application.yml Worker and AI configuration
|   |   |-- src/main/resources/db/migration/ Flyway database migrations
|   |   |   |-- V1__create_processed_claims.sql Initial claims table
|   |   |   |-- V2__add_document_analysis_columns.sql Analysis columns
|   |   |   |-- V3__replace_classification_with_ai_pipeline_columns.sql AI output columns
|   |   |   |-- V4__create_document_guidelines.sql pgvector guidelines and seed data
|   |   |-- build.gradle                 Java dependencies and build tasks
|   |   |-- gradlew                      Gradle wrapper for reproducible builds
|   |   |-- Dockerfile                  Worker build and runtime image
|   |-- 6-ai-extraction-service/        Python OCR and LLM service
|   |   |-- app/main.py                  FastAPI /health and /extract endpoints
|   |   |-- app/config.py                Environment-driven AI settings
|   |   |-- app/extraction.py            PyMuPDF extraction and OCR fallback
|   |   |-- app/redaction.py             spaCy PERSON entity redaction
|   |   |-- app/chunking.py              LangChain document chunking
|   |   |-- app/guidelines.py            Embeddings and pgvector retrieval
|   |   |-- app/llm_client.py            Ollama request and JSON validation flow
|   |   |-- app/schemas.py              Pydantic extraction response schemas
```

	-- requirements.txt	Pinned Python dependencies
	-- Dockerfile	Python runtime, Tesseract, and spaCy model
	-- docker-compose.yml	Local service orchestration
	-- elasticmq.conf	ElasticMQ queue and visibility settings
	-- terraform/	AWS deployment definitions
	-- main.tf	Providers and shared Terraform settings
	-- networking.tf	VPC, subnets, and security groups
	-- api-gateway.tf	API Gateway routes and authorizer
	-- ecs.tf	ECS/Fargate services and task definitions
	-- storage.tf	S3 buckets and IAM storage permissions
	-- sqs.tf	SQS queues and dead-letter queue
	-- rds.tf	Aurora/PostgreSQL resources
	-- security.tf	IAM and security configuration
	-- variables.tf	Deployment input variables
	-- outputs.tf	Terraform deployment outputs
	-- scripts/	Local testing and load utilities
	-- simulate-load.js	Queue/load simulation script
	-- README.md	Architecture and operating documentation

Cloud deployment

The `terraform/` directory contains the AWS-oriented deployment resources. The cloud topology uses API Gateway in front of the gateway service, ECS/Fargate for the gateway and worker, S3 for documents, SQS with a dead-letter queue, Aurora PostgreSQL, IAM roles, and CloudWatch. Configure the required Terraform variables before applying infrastructure, and never commit cloud credentials or Terraform state.